Computer Architecture - Programming Assignment

Tomasulo Multiple-Issue Implementation

Mohammad Sameer

This project extends a Tomasulo simulator to issue up to four instructions per cycle. The main goal was to increase issue width while keeping instructions in program order at issue and commit, and allowing ready instructions to execute out of order.

1. Changes to the implementation

I added an issue-width setting from 1 to 4, with a default of 1. In each cycle, the simulator considers instructions in program order. If the next instruction cannot obtain a reorder buffer (ROB) entry, a reservation station, or a load/store queue slot, issue stops for that cycle. Younger instructions cannot skip past it.

For each issued instruction, source operands are read before its destination register is renamed. RAT updates are applied in issue order, so an instruction can depend on another instruction issued in the same cycle. Register renaming handles WAR and WAW hazards, while producer tags preserve RAW dependencies. Integer register R0 remains zero.

The original Qing (2017) files are kept in `starter/`. The active simulator uses a replacement engine in `code/simulator.py`. I chose this approach because the original code had problems with initialization, mutable state, rename updates, store retirement, cycle advancement, and branch handling that made a simple loop around issue unreliable.

The new engine uses mutable dataclasses, unique increasing ROB tags, a queue of completed results, and tag-based operand wakeup. Instructions retire in program order. The input parser supports hardware settings, register and memory initialization, comments, commas, and case-insensitive instructions, including Beq and Bne.

2. Hardware model

Component	Behavior
Issue	1-4 instructions per cycle, in program order
Writeback and commit	One CDB; at most one instruction commits per cycle
Execution units	Unpipelined integer units and address generators; pipelined FP add and multiply
Memory	One shared, non-pipelined port; configurable access latency
Configuration	Input sets FU and RS counts, ROB size, and execution latencies

3. Pipeline timing and memory handling

An instruction begins execution no earlier than the cycle after issue. A dependent instruction can begin execution after its producer writes back. Loads calculate their address in EX, access data in MEM, and then write back. The start and end cycles shown for EX and MEM are inclusive.

Completed results wait for the single CDB. The result that completed earliest is selected first; ties are resolved by ROB age. Commit occurs after writeback, except that a branch can commit after resolution. Reservation stations are released when EX starts. Load queue entries are released after MEM, while store entries remain until commit. A slot released during a cycle becomes available for issue in the next cycle.

Stores update memory only at the ROB head. A store uses the shared memory port for the configured latency and retires in its final memory cycle. A load forwards from the youngest older store with a matching address, waiting if that store's data is not ready. Forwarding uses the same selection path as other memory work, so it cannot start a second memory operation in the same cycle.

Loads may proceed before all older store addresses are known. If an older store later resolves to an address that conflicts with a load, that load and all younger instructions are replayed. Multiple configured load/store units provide address generators, but they still share one memory port.

4. Branch prediction and recovery

The branch predictor uses an eight-entry, direct-mapped branch target buffer (BTB). It is indexed by the lowest three bits of the word PC. Each entry stores a PC tag, a one-bit prediction, and a target. A branch without a matching entry is predicted not taken. Prediction is used during issue, including within the current issue bundle, and the BTB is updated when the branch resolves in EX.

On a misprediction, the simulator removes younger instructions from the ROB, reservation stations, load/store queue, execution units, memory work, and pending writeback queue. It then rebuilds the RAT from the surviving ROB entries. If resolution occurs in cycle n, cycle n+1 is used for recovery and correct fetching resumes in cycle n+2. Invalid load addresses are checked at retirement, allowing a wrong-path load to be squashed first.

5. Branch offset in the supplied input

The original input uses `Bne R1 R0 -12`, where the starter code interpreted the immediate as a byte displacement. The assignment instead defines the target as:

```
target = PC + 4 + (offset << 2)
```

The runnable `code/test_case.txt` uses -3 to preserve that displacement. This returns to the integer pointer updates rather than the start of the memory loop. The original file remains in `starter/test_case.txt`. A separate case, `tests/vector_loop.in`, uses -8 to run the full three-iteration load/multiply/add/store loop.

6. Results at issue widths 1-4

Each benchmark was run at all four issue widths. Within a benchmark, only issue width changes. Final registers, nonzero memory values, and committed instruction counts were compared with a separate sequential interpreter. Detailed stage histories are saved as text and JSON files under `results/`; the JSON files also record wrong-path instructions.

Test case	Width 1	Width 2	Width 3	Width 4	Speedup
test_case	45	45	45	45	1.000x
independent	20	20	19	19	1.053x
dependencies	21	21	21	21	1.000x
forwarding	18	18	18	18	1.000x
branch_recovery	12	12	12	12	1.000x
memory_replay	22	22	22	22	1.000x
vector_loop	66	65	65	65	1.015x
structural_stalls	13	13	13	13	1.000x

Widths show total cycles. Speedup = cycles at width 1 / cycles at width 4.

The supplied case and vector_loop use integer EX=1, FP add EX=4, FP multiply EX=15, MEM=5, RS counts 4/3/2/5, and ROB=64. The other cases use integer EX=2, FP add EX=3, FP multiply EX=6, MEM=3, eight stations per unit, and ROB=32. The exceptions are memory_replay, with integer EX=7, and structural_stalls, with one integer station and integer EX=4.

Selected final-state checks

Test case	Final values and observations	Count
test_case	R1=0, R2=20; F2=1, F4=3, F6=7, F20=2. Mem[32]=7; Mem[24]=6 and Mem[28]=5 remain unchanged.	15
independent	F4, F6, F8, F10=5; F5, F7, F9, F11=6.	12
dependencies	F4=4, F5=10, F6=6; R1=7, R2=14, R0=0.	8
forwarding	F4=2, F5=3, F6=5; Mem[0]=2, Mem[4]=3.	6
branch_recovery	R1=7, R2=8, Mem[0]=0; wrong-path changes discarded; one misprediction.	3
memory_replay	F4=2, F5=4, Mem[8]=2; one load replay at each width.	4
vector_loop	R1=0, R2=20, F2=3, F20=6; Mem[24]=15, Mem[28]=11, Mem[32]=7; two mispredictions at each width.	25
structural_stalls	F4=5, F5=6; the blocked second integer instruction prevents younger FP issue.	4

7. What the results show

Increasing issue width gives only a small improvement on these benchmarks. The independent case drops from 20 to 19 cycles, a speedup of about 1.053x. The vector loop drops from 66 to 65 cycles, or about 1.015x. The remaining cases finish in the same number of cycles at all four widths.

Wider issue does not make every stage wider. The simulator still has one CDB, one commit per cycle, and one shared memory port. Dependencies and long FP operations also limit progress. As a result, issuing instructions earlier often does not change when the last instruction commits.

8. Correctness checks

The test suite contains 16 tests. These cover same-cycle RAW, WAR, and WAW dependencies; store forwarding; replay after a late store address; branches and loops; BTB index conflicts; RAT restoration; and cancellation of wrong-path work. They also check wrong-path stores and invalid loads, n+2 recovery, in-order issue stalls, ROB and LSQ capacity limits, memory-port use, execution timing, multiple functional units, parser validation, issue-width validation, and R0.

One test generates 25 seeded random programs and runs each at four issue widths, giving 100 comparisons with the sequential interpreter. Additional checks cover the single CDB, retirement order, execution latency, issue width, and stage boundaries. The 32 saved benchmark runs also match the interpreter.

9. Limitations

Arithmetic uses Python integers and floats, so bit-level overflow and single-precision IEEE-754 rounding are not modeled. Memory stores word values by their starting byte address; overlapping unaligned words do not share individual bytes. Four-byte words may start at addresses 0 through 252 in the 256-byte memory.

The simulator keeps dynamic instruction histories, which increases host memory use for long programs. The `--max-cycles` option stops runaway execution. These tests cover the implemented behavior, but they do not establish correctness for every possible program.

10. Running the project

Use Python 3.10 or later. No packages beyond the standard library are required. Run the following commands from the project directory:

```
python code/main.py
python tests/test_simulator.py
python run_benchmarks.py
python code/main.py tests/vector_loop.in --issue-width 4
```

Attribution. This project is based on the supplied Qing (2017) code and the assignment specification. The sample report was used as a guide for organization. Its in-order execution and limited branch support were not adopted. Setup instructions are also provided in README.md.